

Adapter lab

Trabalho de Laboratório — Padrão Adapter em Java

Sistema de documentos HTML e PDF

Objetivo

Implementar, em Java, um pequeno sistema usando o padrão **Adapter**.

O objetivo é mostrar como uma classe já existente, com uma interface incompatível, pode ser integrada a um sistema sem modificar o código cliente.

O modelo segue a estrutura do trabalho de laboratório sobre Template Method anexado.

Contexto

Considere um sistema web de venda de veículos.

Esse sistema já trabalha com documentos usando a interface:

```
interface Documento {  
    void setConteudo(String conteudo);  
    void desenha();  
    void imprime();  
}
```

Já existe uma implementação para documentos HTML:

```
class DocumentoHtml implements Documento
```

Depois, a empresa decide incluir documentos PDF.

Porém, o componente PDF já existente possui outra interface:

```
class ComponentePdf {  
    void pdfFixaConteudo(String conteudo);  
    void pdfPreparaExibicao();  
    void pdfAtualiza();  
    void pdfFinalizaExibicao();  
}
```

```
void pdfEnviaImpressora();  
}
```

O problema é que ComponentePdf **não implementa** a interface Documento.

Para resolver isso, deve ser criada uma classe adaptadora:

```
class DocumentoPdf implements Documento
```

Ela deverá conter um objeto ComponentePdf e traduzir as chamadas da interface Documento para chamadas compatíveis com o componente PDF.

Estrutura esperada

Interface

Documento

Classes

DocumentoHtml

ComponentePdf

DocumentoPdf

ServidorWeb

Parte 1 — Interface Documento

Implemente a interface Documento com os métodos:

```
void setConteudo(String conteudo);  
void desenha();  
void imprime();
```

Essa interface representa o tipo esperado pelo sistema cliente.

Parte 2 — Classe DocumentoHtml

Implemente a classe DocumentoHtml, que implementa Documento.

Ela deve possuir um atributo:

```
protected String conteudo;
```

Comportamento esperado:

```
setConteudo(conteudo)
```

deve armazenar o conteúdo.

```
desenha()
```

deve mostrar:

Desenha documento HTML: <conteudo>

```
imprime()
```

deve mostrar:

Imprime documento HTML: <conteudo>

Parte 3 — Classe ComponentePdf

Implemente a classe ComponentePdf.

Essa classe representa um componente externo já existente.

Ela **não deve implementar** a interface Documento.

Ela deve possuir os métodos:

```
void pdfFixaConteudo(String conteudo);
```

```
void pdfPreparaExibicao();
```

```
void pdfAtualiza();
```

```
void pdfFinalizaExibicao();
```

```
void pdfEnviaImpressora();
```

Comportamento esperado:

Exibição PDF: início

Exibição conteúdo PDF: <conteudo>

Exibição PDF: fim

Impressão PDF: <conteudo>

Parte 4 — Classe DocumentoPdf

Implemente a classe DocumentoPdf.

Ela deve:

implements Documento

e deve possuir um atributo:

protected ComponentePdf ferramentaPdf;

A classe DocumentoPdf é o **Adapter**.

Ela deve traduzir os métodos da interface Documento para métodos da classe ComponentePdf.

Mapeamento esperado:

setConteudo(conteudo)

deve chamar:

ferramentaPdf.pdfFixaConteudo(conteudo);

desenha()

deve chamar:

ferramentaPdf.pdfPreparaExibicao();

ferramentaPdf.pdfAtualiza();

ferramentaPdf.pdfFinalizaExibicao();

imprime()

deve chamar:

ferramentaPdf.pdfEnviaImpressora();

Parte 5 — Classe ServidorWeb

Crie a classe ServidorWeb.

Ela deve:

1. Criar um documento HTML.
2. Definir seu conteúdo.
3. Desenhar o documento HTML.
4. Imprimir o documento HTML.
5. Criar um documento PDF usando DocumentoPdf.
6. Definir seu conteúdo.
7. Desenhar o documento PDF.
8. Imprimir o documento PDF.

O programa deve usar variáveis do tipo Documento, por exemplo:

```
Documento doc1 = new DocumentoHtml();
```

```
Documento doc2 = new DocumentoPdf();
```

Entrega

O aluno deverá entregar:

1. O Diagrama de classes UML. O diagrama UML deve mostrar:

Documento

DocumentoHtml

DocumentoPdf

ComponentePdf

ServidorWeb

Também deve mostrar:

- DocumentoHtml implementando Documento;
- DocumentoPdf implementando Documento;
- associação entre DocumentoPdf e ComponentePdf;
- ServidorWeb usando objetos do tipo Documento.

Perguntas adicionais

1. Qual classe representa o Adapter?

2. Qual classe representa o componente adaptado?
 3. Qual interface é esperada pelo sistema cliente?
 4. Por que ComponentePdf não pode ser usado diretamente como Documento?
 5. Onde ocorre a tradução entre as duas interfaces?
 6. Por que DocumentoPdf implementa Documento?
 7. Qual é a vantagem de o ServidorWeb usar variáveis do tipo Documento?
 8. Por que este projeto é um exemplo do padrão Adapter?
 9. Qual a diferença principal entre Adapter e Strategy?
 10. O código cliente precisou ser alterado para conhecer os métodos pdf...?
-

Restrições

- Não alterar a classe ComponentePdf para implementar Documento.
- O cliente ServidorWeb deve usar a interface Documento.
- A classe DocumentoPdf deve ser responsável pela adaptação.
- Não usar if ou switch para diferenciar HTML e PDF.
- O código deve compilar e executar corretamente.
- Usar nomes coerentes e boa indentação.

Saída esperada

Desenha documento HTML: Hello

Imprime documento HTML: Hello

Exibição PDF: início

Exibição conteúdo PDF: Bonjour

Exibição PDF: fim

Impressão PDF: Bonjour

Preparar o código que será apresentado no laboratório mas que não deve ser mandado no trabalho !